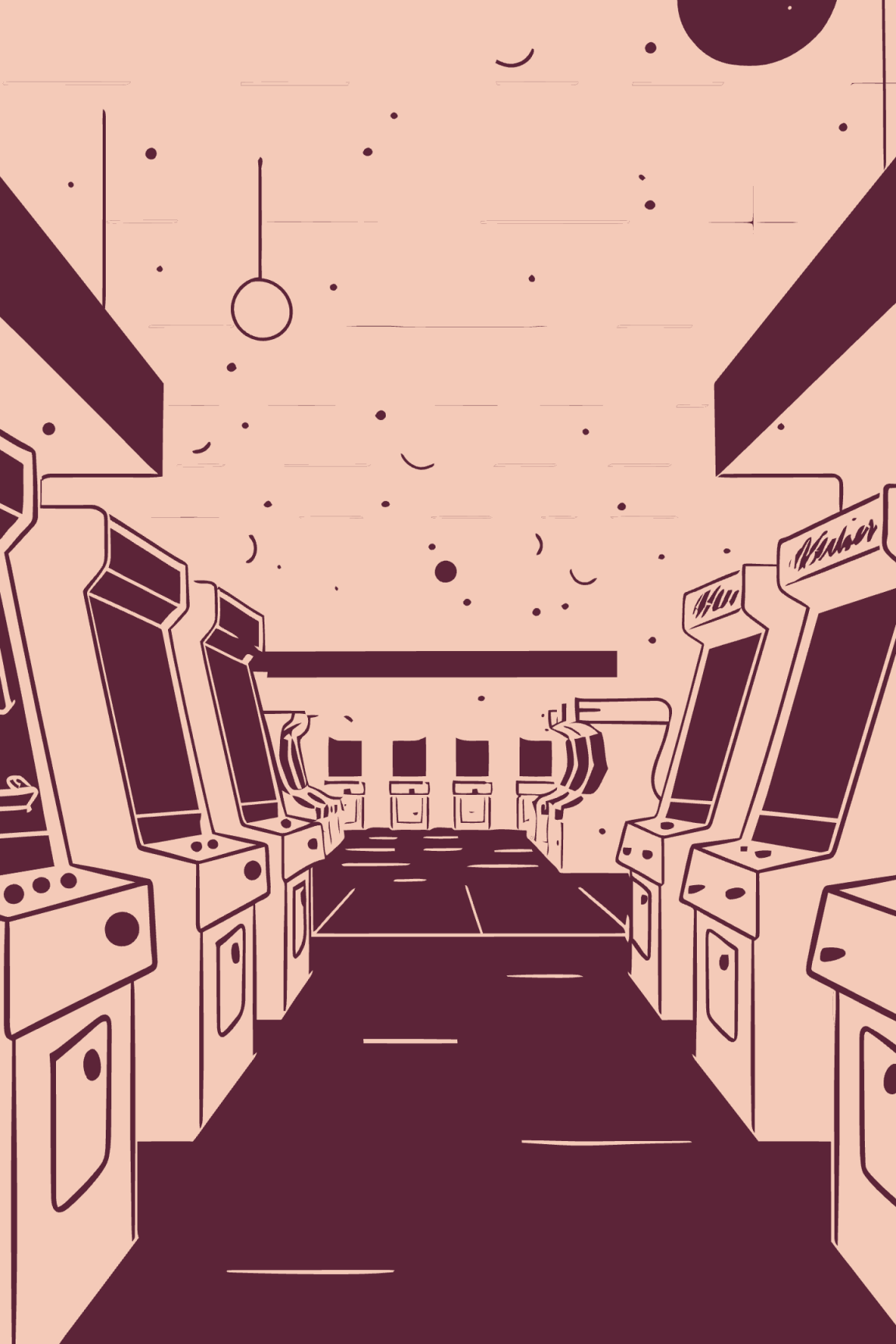




Intelligent Supply Chain Automation

A 3-layer autonomous system that transforms reactive logistics into a self-optimising, predictive operation — sensing, thinking, and acting in real time.

PROBLEM DEFINITION & SOLUTION ARCHITECTURE

The Supply Chain Problem

Modern supply chains are under mounting pressure — yet most still rely on fragmented data, manual decision-making, and reactive responses. The gap between what operations demand and what legacy systems deliver has never been wider.

No Real-Time Visibility

Inventory levels, vehicle locations, and supplier statuses are updated hours or days after the fact. Managers are perpetually flying blind, making decisions based on stale data that no longer reflects ground reality.

Reactive, Not Predictive

Organisations respond to disruptions only after they occur — a stock-out is discovered after a customer complaint, a delay is flagged after a shipment misses its window. There is no forward-looking intelligence to anticipate and prevent issues.

Manual Decision-Making

Critical decisions — reordering, rerouting, switching suppliers — rely on individual judgement under time pressure. This is slow, inconsistent, and prone to costly human error, especially during high-demand or crisis periods.

Delays, Stockouts & High Costs

The downstream effects are severe: missed delivery windows, excess safety stock tying up capital, customer churn, and operational overhead that erodes margins. These are symptoms of a fundamentally broken feedback loop.

The Proposed Solution: A 3-Layer Intelligent System

Rather than patching individual pain points, the proposed architecture addresses the root cause — the absence of closed-loop intelligence. The system is designed as three tightly integrated layers, each with a distinct responsibility, forming a continuous sense-think-act cycle.



Layer 1: IoT — Sense

Continuously collects real-time data from inventory sensors, GPS-tracked vehicles, environmental monitors, and external demand signals. This layer is the system's nervous system — always on, always streaming.



Layer 2: AI — Think

Processes and analyses incoming data streams to generate demand forecasts, predict disruptions, and optimise routing decisions. Machine learning models surface actionable recommendations at machine speed, without waiting for human review.



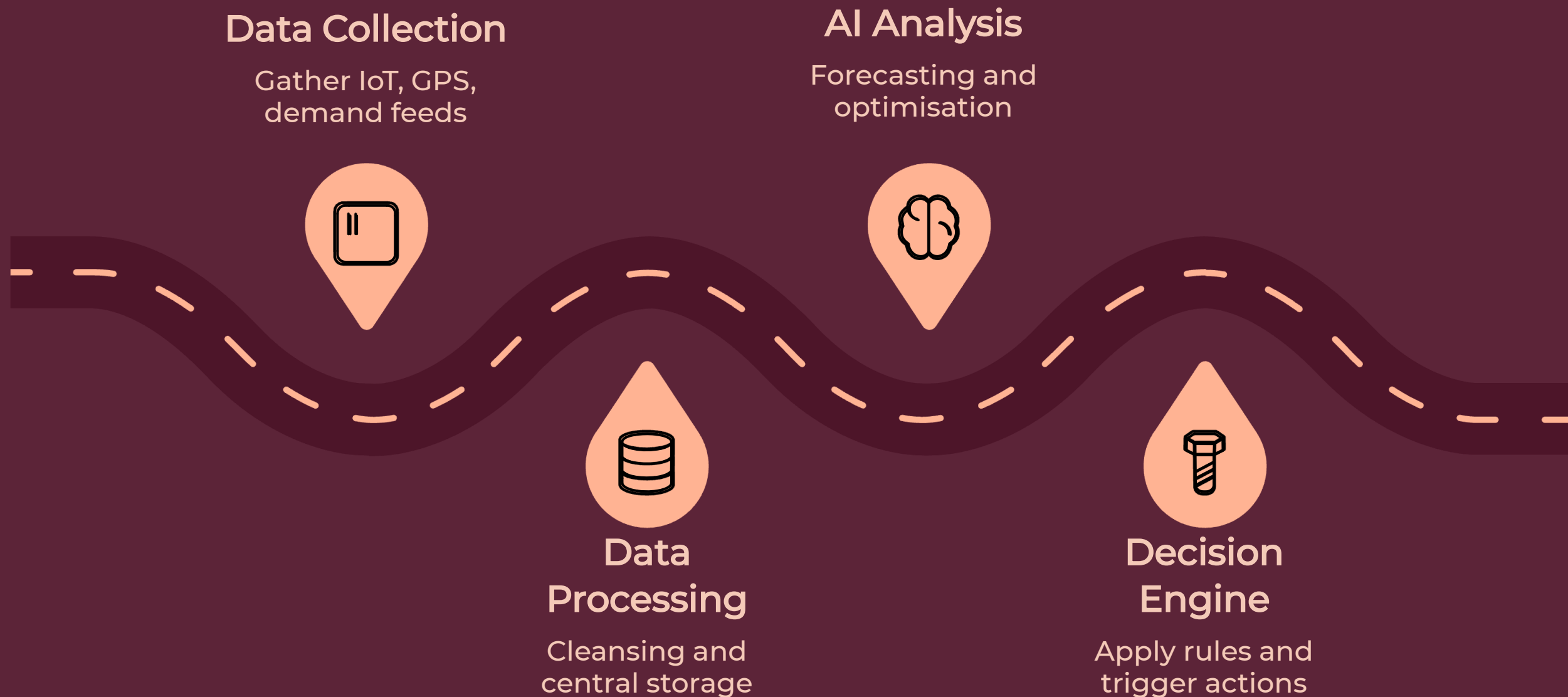
Layer 3: Autonomous — Act

Translates AI recommendations into concrete actions — automatically placing reorders, switching suppliers, or rerouting vehicles. No human bottleneck. Actions are logged, traceable, and auditable in real time via the Control Tower dashboard.

- Each layer is independently scalable, yet designed for seamless interoperability. The result is a supply chain that doesn't just report what happened — it anticipates what will happen and acts before the window closes.

End-to-End Workflow: From Sensor to Action

The following six-step workflow describes how raw physical-world data becomes an autonomous supply chain decision — from the moment a sensor fires to the moment an order is placed or a route is changed.



Each step is designed to minimise latency. From IoT event to autonomous action, the system operates in near real time, ensuring that supply chain disruptions are addressed within minutes rather than hours or days.

Steps 1 & 2: Data Collection & Processing

Step 1 — Data Collection (IoT Layer)

The IoT layer acts as the system's sensory network, continuously capturing live signals from across the physical supply chain.

- **Inventory Sensors** — Monitor real-time stock levels at warehouses and distribution centres, triggering alerts when thresholds are breached.
- **GPS Tracking** — Provides continuous vehicle location and movement data, enabling live route monitoring and deviation detection.
- **External Inputs** — Ingests demand trend signals from market feeds, POS systems, and third-party APIs to enrich the internal data picture with external context.

Step 2 — Data Processing

Raw sensor data is noisy, incomplete, and heterogeneous. Before any AI analysis can take place, the data must be cleaned, normalised, and structured for reliable downstream consumption.

- **Cleansing & Normalisation** — Removes duplicates, corrects sensor anomalies, fills gaps using interpolation, and converts all data to a consistent schema.
- **Central Data Store** — All processed data lands in a unified database or in-memory store, providing a single source of truth accessible by the AI and decision layers simultaneously.
- **Latency Targets** — Processing pipelines are optimised to keep end-to-end latency from sensor event to stored record under a defined SLA, typically seconds rather than minutes.



AI-Driven Autonomous Supply Chain Control Tower

Sense Layer – IoT & Data Simulation

REAL-TIME VISIBILITY

DIGITAL TWIN

SCALABLE ARCHITECTURE



The Supply Chain is Flying Blind

Modern supply chains are plagued by poor visibility, reactive processes, and mounting inefficiencies that cost time and money.

No Real-Time Visibility

Goods move without live tracking, leaving managers guessing shipment status until it's too late.

Inventory Imbalance

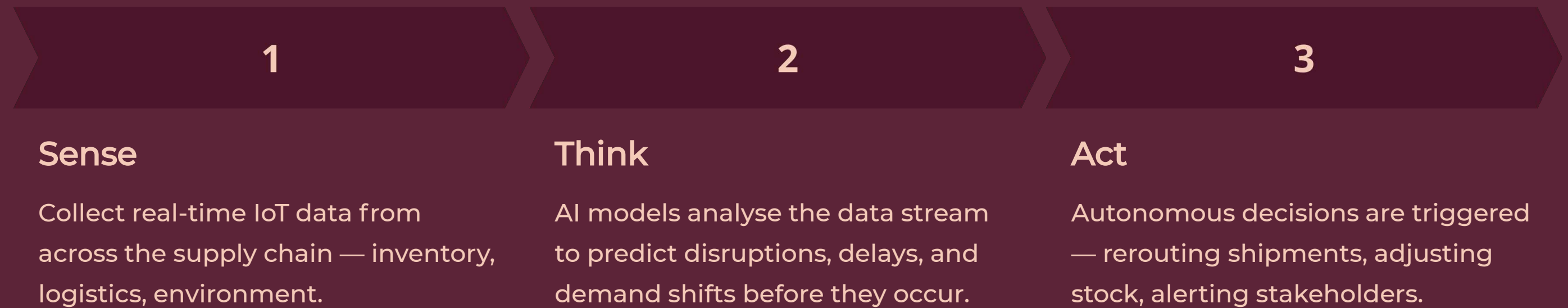
Overstocking ties up capital; stockouts kill customer trust. Both stem from the same root — bad data.

Reactive, Not Proactive

Current systems respond to disruptions after they happen, rather than anticipating and preventing them.

Our Solution: Sense → Think → Act

A three-layer AI-driven control tower that transforms raw sensor data into autonomous, intelligent supply chain decisions.



 **Our Focus:** We are responsible for the **Sense Layer** — the data foundation that makes everything else possible.

What is the Sense Layer?



The Sense Layer acts as the **nervous system** of the control tower — continuously capturing physical-world events and converting them into structured digital data.

Inventory

Stock levels, inflow & outflow rates

Logistics

GPS coordinates, speed, route status

Environment

Temperature, humidity, condition flags

Data We Simulated

Synthetic IoT streams generated in Python simulate three parallel data domains — updating every few seconds to mirror real-world supply chain behaviour.



Inventory Stream

Stock level, units inflow, units outflow — tracked per warehouse location in real time.



Logistics Stream

Latitude, longitude, and speed for each truck — enabling live route and ETA monitoring.

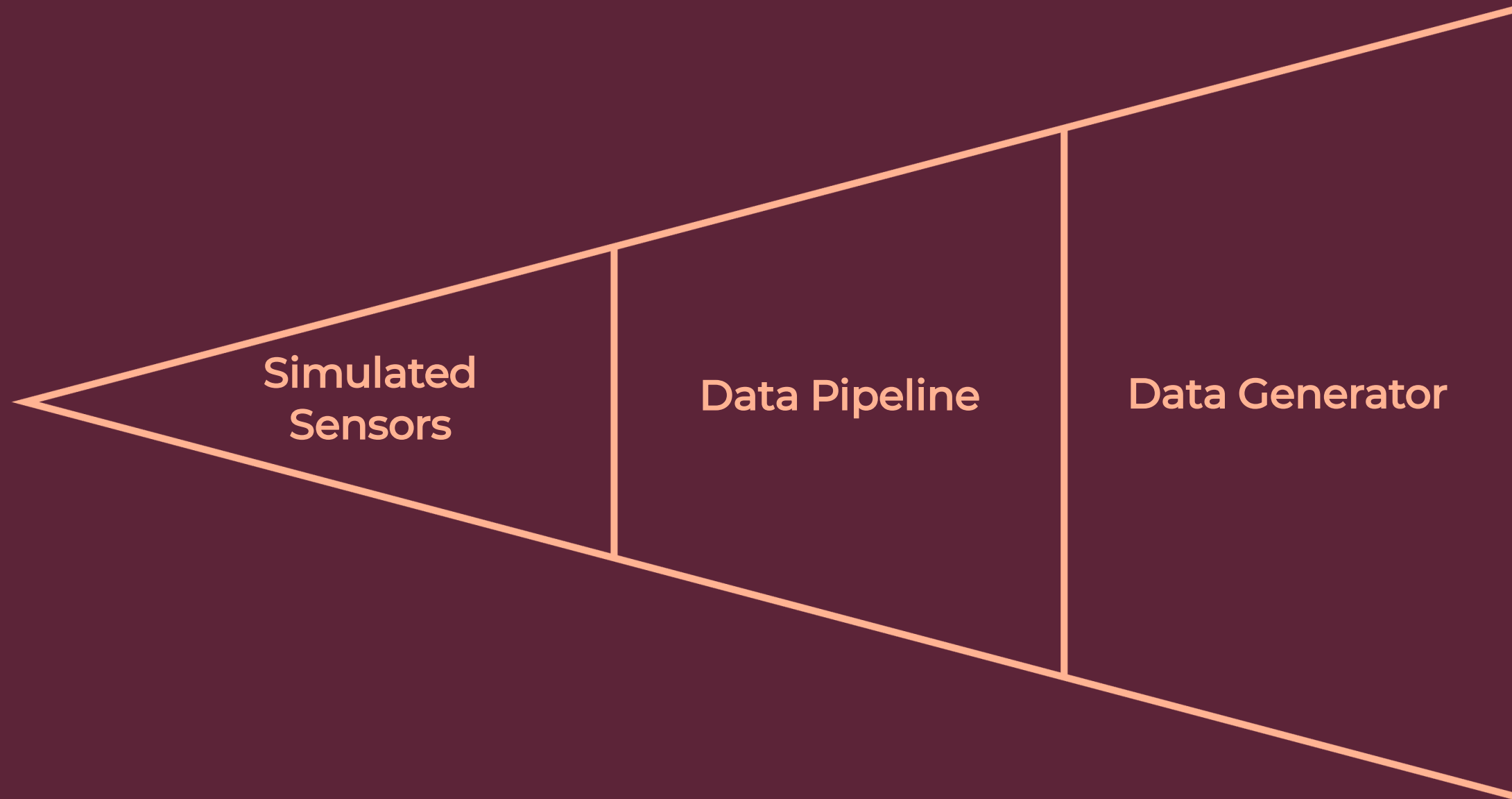


Truck Status

Binary status flags — **In Transit** or **Stopped** — with timestamps for dwell-time analysis.

  **Data refreshes every 5 seconds** — mimicking a genuine real-time IoT simulation environment.

System Architecture



Each layer is independently modular — sensors can be swapped for real hardware, and the pipeline can adopt MQTT or Kafka streams without redesigning downstream components.

Data Pipeline Implementation



How the Pipeline Works

Synthetic IoT data is generated using Python scripts, structured into a consistent schema, and exported via **CSV files** or a **REST API** for downstream consumption.

→ Scalable by Design

Modular architecture allows adding new data sources — warehouses, ports, suppliers — without rebuilding the pipeline.

→ Production-Ready Path

Replace Python simulators with real sensors and swap CSV outputs for live Kafka topics — zero structural changes required.



Real-Time Simulation in Action

5s

Refresh Interval

Data updates every 5 seconds, creating a continuous live feed

3

Data Streams

Inventory, logistics, and truck status running in parallel

1

Unified Dashboard

All streams converge into a single real-time visualisation layer

Key Features at a Glance



Real-Time Simulation

Continuous data generation at 5-second intervals mirrors live IoT sensor behaviour.



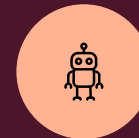
Unified Dataset

Inventory and logistics streams are combined into a single, AI-ready data schema.



Scalable Architecture

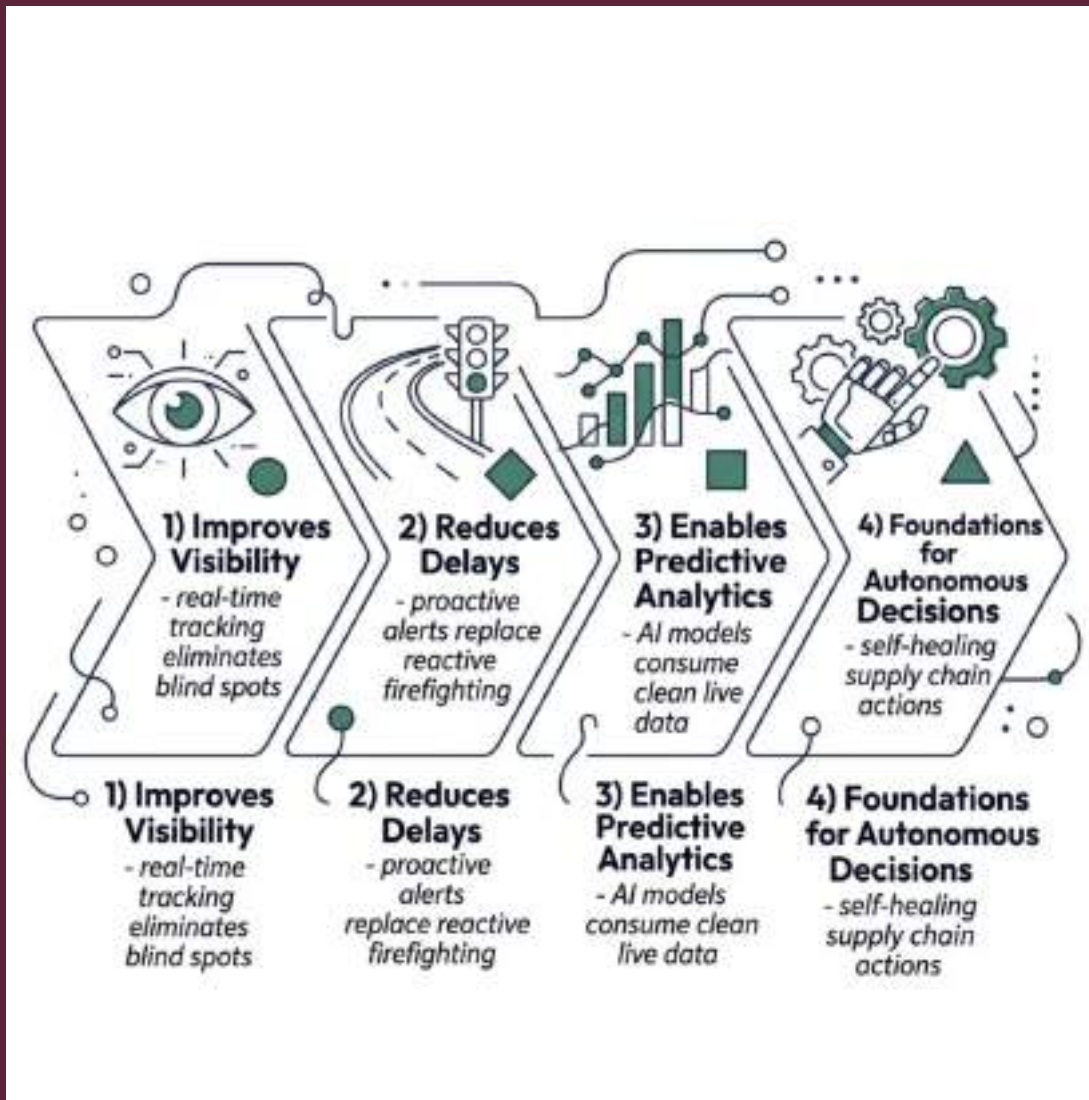
Modular design supports growth — new warehouses, sensors, and regions plug in seamlessly.



AI Integration Ready

Structured, consistent data format feeds directly into predictive analytics and ML models.

From Data to Intelligence to Automation



The Road Ahead

The Sense Layer is the first and most critical step. Future enhancements will deepen its capabilities:

- "From data → intelligence → automation" — the Sense Layer makes this journey possible.

01

Real IoT Sensors

Replace simulated data with hardware sensors on trucks, shelves, and cold-chain assets.

02

MQTT / Kafka Streaming

Upgrade the pipeline to industry-standard event streaming for sub-second latency.

03

Multi-Warehouse Scale

Extend the digital twin across global distribution networks with unified monitoring

Step 3: AI Analysis — The Intelligence Engine

The AI layer is the cognitive core of the system. It continuously interrogates the processed data to generate three categories of output: demand forecasts, disruption predictions, and optimised routing plans. These outputs feed directly into the autonomous decision engine.

Demand Forecasting

Using time-series models and machine learning algorithms, the system predicts future demand by SKU, location, and time horizon. It accounts for seasonal patterns, promotional calendars, external market signals, and historical consumption rates. Forecasts are continuously updated as new data arrives, ensuring predictions remain accurate even in volatile conditions.

Delay Prediction

The AI analyses GPS telemetry, route conditions, weather data, and supplier lead-time histories to predict where delays are likely to occur before they materialise. Risk scores are assigned to each active shipment, allowing the system to prioritise intervention where it matters most.

Route Optimisation

Multi-constraint optimisation algorithms evaluate thousands of possible routing configurations in real time, balancing delivery time, fuel cost, vehicle capacity, and customer priority. The recommended route is passed directly to the autonomous execution layer for immediate implementation.

AI Analysis — The Intelligence Engine

Converting real-time supply chain data into **automatic actions** — enabling a proactive, self-operating logistics system that responds to disruptions before they escalate.



How It Works



Three-Layer Intelligence

The engine sits at the intersection of data ingestion and action execution, processing inputs continuously and responding in real time.



IoT Layer

Live feeds from inventory sensors and truck status trackers



AI Layer

Demand forecasting and delay prediction models



Decision Logic

Rule-based triggers translate insights into executable actions

Key Automated Decisions

Three core decision types keep the supply chain running without manual intervention.



Auto Reorder

Triggered automatically when inventory levels fall below a pre-set threshold ensuring stock continuity without human oversight.



Route Optimisation

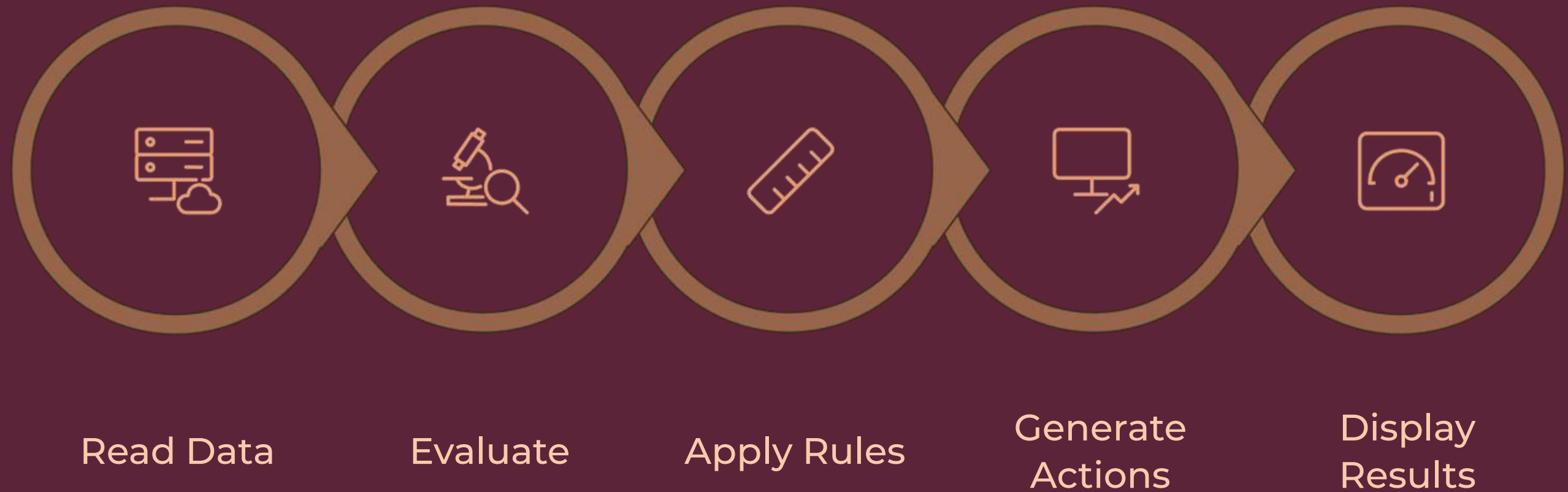
Activated the moment delivery delays are detected, rerouting vehicles to the fastest available alternate path.



Supplier Switching

Under critical delay conditions, the engine seamlessly activates a backup supplier to maintain fulfilment continuity.

Decision Workflow



Every cycle runs autonomously and continuously meaning, no manual input required at any stage. The engine evaluates conditions and executes decisions in milliseconds, keeping the supply chain one step ahead.

Sample Output in Action

Stock Replenishment

Order automatically placed for the required number of units upon threshold breach

Route Adjustment

Delivery rerouted to an alternate path, minimising delay impact on end customers

Supplier Activation

Backup supplier selected and engaged without any manual escalation or approval

Key Features & Value



Real-Time Decision-Making

Processes live data streams and triggers actions with zero latency — critical for time-sensitive logistics events.



Fully Autonomous Operation

No manual input required at any stage. The engine acts independently across reorder, routing, and supplier decisions.



Improved Efficiency & Responsiveness

Reduces response times, minimises disruption impact, and frees operations teams to focus on strategic priorities.

"Enables the system to act instantly on supply chain disruptions using automated decision logic."



The Automated Decision Engine is the operational backbone of a self-healing supply chain — bridging the gap between insight and action at machine speed.

This is the implementation of our autonomous decision engine. It takes real-time data, evaluates conditions like stock levels and delays, and automatically triggers actions such as reordering inventory and rerouting deliveries.

```
import pandas as pd

demand_data = pd.read_csv("demand_data.csv")
iot_data = pd.read_excel("combined_iot_data.xlsx")
latest_demand = demand_data.iloc[-1]
latest_iot = iot_data.iloc[-1]
stock = latest_demand["Inventory"]
predicted_demand = latest_demand["Demand"]

truck_status = latest_iot["truck_status"]
route = latest_iot["route_id"]
if truck_status == "Stopped":
    delay = 15
else:
    delay = 5
    actions = []

#Auto reorder
actions = []

if stock < 400:
    order_qty = predicted_demand - stock
    actions.append(f"Order placed for {order_qty} units")
if delay > 10:
    actions.append(f"Route changed from {route} to alternate route")
if delay > 12:
    actions.append("Backup supplier selected")
print("Autonomous Actions")
for action in actions:
    print(action)

print("Stock value:", stock)|
```

Steps 4 & 5: Autonomous Decision Engine & Execution

This is where recommendations become reality. The Autonomous Decision Engine applies a structured rule set to AI outputs, converting insights into concrete, pre-authorised actions — without waiting for human approval. The distinction between a system that *suggests* and one that *acts* is what separates genuine automation from glorified dashboards.

Step 4 — Autonomous Decision Engine

The engine evaluates each AI output against a configurable rule matrix. Rules encode business logic — reorder thresholds, approved supplier lists, rerouting triggers — and govern which actions are taken automatically versus which are escalated for human review.

Auto Reorder

When forecast demand exceeds projected stock, a purchase order is automatically generated and submitted to the preferred supplier.

Route Switch

When delay risk exceeds a threshold, the optimal alternative route is activated and the driver is notified immediately.

Supplier Change

If a primary supplier is flagged as unreliable or unavailable, the system automatically pivots to a pre-approved secondary source.

Step 5 — Execution Layer

Once a decision is made, the execution layer carries it out across integrated systems. Every action is logged with a full audit trail — timestamp, triggering condition, AI confidence score, and outcome — ensuring full traceability and compliance readiness.

- **Order Placed** — Purchase order sent to supplier via API or EDI integration
- **Route Updated** — New routing instructions dispatched to driver mobile or fleet management system
- **Inventory Adjusted** — Stock records updated in the warehouse management system to reflect pending inbound orders

Simulation Scenario: Sudden Demand Spike

To validate the system's behaviour under stress, the following scenario illustrates how the full sense-think-act loop responds to one of the most common and costly supply chain disruptions — an unexpected surge in product demand.

Inventory Drops

A rapid increase in order volume causes warehouse stock to fall below the safety threshold faster than anticipated by the weekly planning cycle.

AI Predicts Demand Surge

The AI model correlates the stock event with external demand signals — social trends, regional sales velocity, seasonal uplift — and predicts a sustained demand spike over the next 72 hours.

Dashboard Shows Action

The Control Tower dashboard surfaces a real-time alert, logs the autonomous action taken, and updates the projected inventory timeline — giving operators full visibility with zero manual intervention required.

IoT Detects Low Stock

Shelf sensors register the declining inventory level and immediately push a low-stock event to the data processing pipeline, timestamped to the second.

System Auto-Orders

The Autonomous Decision Engine triggers an emergency reorder at the optimal quantity, selecting the fastest available supplier from the approved list and submitting the purchase order automatically.

 **Key Insight:** From the moment the IoT sensor fires to the moment the purchase order is submitted, the entire cycle completes in under 60 seconds — compared to hours or days in a manual workflow. This is the operational advantage of closed-loop autonomy.

Prototype Architecture & Technology Stack

The prototype is built to demonstrate the full sense-think-act loop in a controlled simulation environment. The technology stack is deliberately lean and open-source, ensuring accessibility for evaluation teams while remaining production-scalable for enterprise deployment.

⚙️ Backend — Python

Python serves as the core runtime environment, handling all business logic, AI model execution, and automation rule processing. Key libraries include:

- **Pandas / NumPy** — Data manipulation and numerical processing
- **Scikit-learn / Prophet** — Demand forecasting and predictive modelling
- **OR-Tools** — Route optimisation and constraint solving
- **FastAPI** — Lightweight API layer connecting backend to dashboard

📁 Data Simulation Layer

In the absence of live IoT hardware, the prototype generates realistic data streams to simulate real-world supply chain conditions.

- **CSV Files** — Static datasets representing historical inventory, demand, and shipment records for model training
- **Random Data Generation** — Python scripts simulate continuous sensor events, including noise and anomalies, to stress-test AI responses
- **External APIs** — Optional integration with public weather, traffic, or market data APIs to enrich simulation realism

💻 Dashboard — Streamlit

Streamlit provides a rapid-development web UI that renders the Control Tower experience without requiring a dedicated front-end engineer.

- **Deployment** — Runs locally or on Streamlit Cloud for remote access during evaluations
- **Interactivity** — Sliders and dropdowns allow evaluators to simulate demand spikes and observe system responses live
- **Refresh Rate** — Dashboard polls the backend every few seconds to maintain near real-time data fidelity

Control Tower Dashboard: Full Operational Visibility

The dashboard is the human interface to an autonomous system — not a tool for making decisions, but for monitoring, validating, and overriding them. It provides operations teams with complete situational awareness across all active supply chain processes.



Inventory Levels

Live stock position by SKU and location, with colour-coded status indicators (healthy / at-risk / critical). Projected stock-out dates are displayed alongside auto-reorder confirmations, so operators can verify autonomous actions at a glance.



Vehicle Tracking

An optional live map view displays all active shipments with real-time GPS position, estimated arrival time, and delay risk score. Route changes made autonomously are highlighted, with before-and-after comparison available on click.



Alerts & Notifications

Configurable alert thresholds trigger instant notifications for low stock events, predicted delays, supplier risk flags, and system anomalies. Alerts are ranked by severity and linked directly to the relevant autonomous action or escalation workflow.



Autonomous Actions Log

Every autonomous decision is recorded in a structured audit log — capturing the triggering condition, AI confidence score, action taken, and outcome. This log is critical for compliance, continuous model improvement, and building operator trust in the system's judgement over time.

The Think Layer

Converting raw sensor data into smart decisions. The Think Layer sits at the core of the pipeline — bridging real-world sensor input with actionable logistics outputs using supervised learning and graph algorithms.

0 1 • Demand Prediction

Predict future product demand from inventory and sales history.

Model: Linear Regression

0 2 • Route Optimization

Find the lowest-cost delivery path factoring distance, traffic, and risk.

Algorithm: Dijkstra + Weighted Cost

0 3 • Risk & Delay Prediction

Classify whether a delivery will face delays before dispatch.

Model: Random Forest Classifier



Demand Prediction Model

Linear Regression · Supervised Learning — Forecasts future product demand to prevent stockouts and overstocking, automatically triggering reorder signals to the ACT layer.

 Input Features (X) → Target (Y)

Day	Sales	Inventory	Price	Demand
1	100	500	50	110
2	120	450	52	130
3	90	400	48	95

 Output → Predicted Demand: 115.7 units"Stock will run out in 2 days — trigger reorder"

 Source Code

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

data = pd.read_csv("demand_data.csv")
X = data[['Sales', 'Inventory', 'Price']]
y = data['Demand']

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.2)

model = LinearRegression()
model.fit(X_train, y_train)

new_data = pd.DataFrame(
    [[110, 420, 51]],
    columns=['Sales', 'Inventory', 'Price']
)

predicted_demand = model.predict(new_data)
print("Predicted Demand:", predicted_demand[0])
```


Route Optimization Model

Dijkstra's Algorithm + Weighted Cost Function — Traverses a graph of delivery nodes and selects the lowest-cost path by weighing distance, traffic, and risk. Weights are tunable to simulate adaptive AI intelligence.

Input Routes

From	To	Distance	Traffic	Risk	Cost
A	B	10	2	1	30
A	C	15	1	2	40
B	D	20	3	2	55
C	D	10	1	1	25

 **Cost Formula:** Distance + (Traffic × 5) + (Risk × 10)

 **Best Path:** A → C → D **Total Cost:** 80 | **ETA:** 42 min

Source Code

```
import heapq

def dijkstra(graph, start):
    costs = {node: float('inf')}
    for node in graph:
        costs[node] = 0
    queue = [(0, start, [start])]

    while queue:
        cost, node, path = \
            heapq.heappop(queue)
        for neighbor, weight \
            in graph[node].items():
            new_cost = cost + weight
            if new_cost < costs[neighbor]:
                costs[neighbor] = new_cost
                heapq.heappush(queue,
                    (new_cost, neighbor,
                     path + [neighbor]))
    return costs

# Weighted graph (cost per edge)
graph = {
    'A': {'B': 30, 'C': 40},
    'B': {'D': 55},
    'C': {'D': 25},
    'D': {}
}
print(dijkstra(graph, 'A'))
```

Risk & Delay Prediction Model

Random Forest Classifier — Evaluates six delivery features to classify whether a route will experience a delay. Enables proactive rerouting before incidents occur.

Input Features

Feature	Description	Range
Distance	Route length (km)	10 – 50
Traffic	Congestion level	1 – 5
Weather	Clear / Bad	0 or 1
Load	Truck load %	0 – 100
Time	Hour of day	0 – 23
RiskScore	From route model	0 – 20



Logic Rules: High traffic + Bad weather → High delay risk
Heavy load + Long distance → Higher risk score



Sample Input → [20, 4, 1, 75, 17, 18]
Output → **Delay Prediction: YES** ⚠️ "High delay risk detected — trigger route switch"

Source Code

```
import pandas as pd
from sklearn.ensemble import \
    RandomForestClassifier
from sklearn.model_selection import \
    train_test_split

data = pd.DataFrame({
    'Distance': [10,20,15,30,25],
    'Traffic': [2, 4, 3, 5, 1],
    'Weather': [0, 1, 0, 1, 0],
    'Load': [50,80,60,90,40],
    'Time': [10,18,14,20, 9],
    'RiskScore': [5,15,10,20, 3],
    'Delay': [0, 1, 0, 1, 0]
})

X = data[['Distance','Traffic','Weather',
    'Load','Time','RiskScore']]
y = data['Delay']

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.2)

model = RandomForestClassifier()
model.fit(X_train, y_train)

new_data = pd.DataFrame(
    [[20,4,1,75,17,18]],
    columns=['Distance','Traffic','Weather',
    'Load','Time','RiskScore'])

prediction = model.predict(new_data)
print("Delay:",
    "YES" if prediction[0]==1 else "NO")
```

SMARTFACTORY DASHBOARD

Sense . Think Act

An exploration of modern factory telemetry, 3D digital twins, and AI-driven insights.

1. CORE ARCHITECTURE

Navigating the High-Performance Smart Factory Interface

DASHBOARD OVERVIEW

INTERFACE & EXPERIENCE

- The Smart Factory UI is engineered with a high contrast, dark aesthetic to minimize eye strain while highlighting critical operational data points.
- A persistent Header displays the Global State, Live Clock, and seamlessly toggles between 'Simulation' and 'Real Data' modes.
- Modular navigation allows operators to rapidly switch contexts between Live Operations and Deep AI Analytics without breaking workflow.

KPI METRICS PANEL

04

Core Metrics Tracked

REAL-TIME TELEMETRY

The dashboard dynamically calculates and visualizes four critical parameters: Production Output, System Efficiency, Downtime Events, and Energy Consumption (kWh).

It computes complex ratios on the fly, leveraging live inflows and outflows to maintain peak factory optimization.

$$\text{Efficiency} = \frac{\Sigma \text{ Inflow}}{\Sigma \text{ Outflow}} * 100$$

2. 3D DIGITAL TWIN

Powered by Three.js Visualization

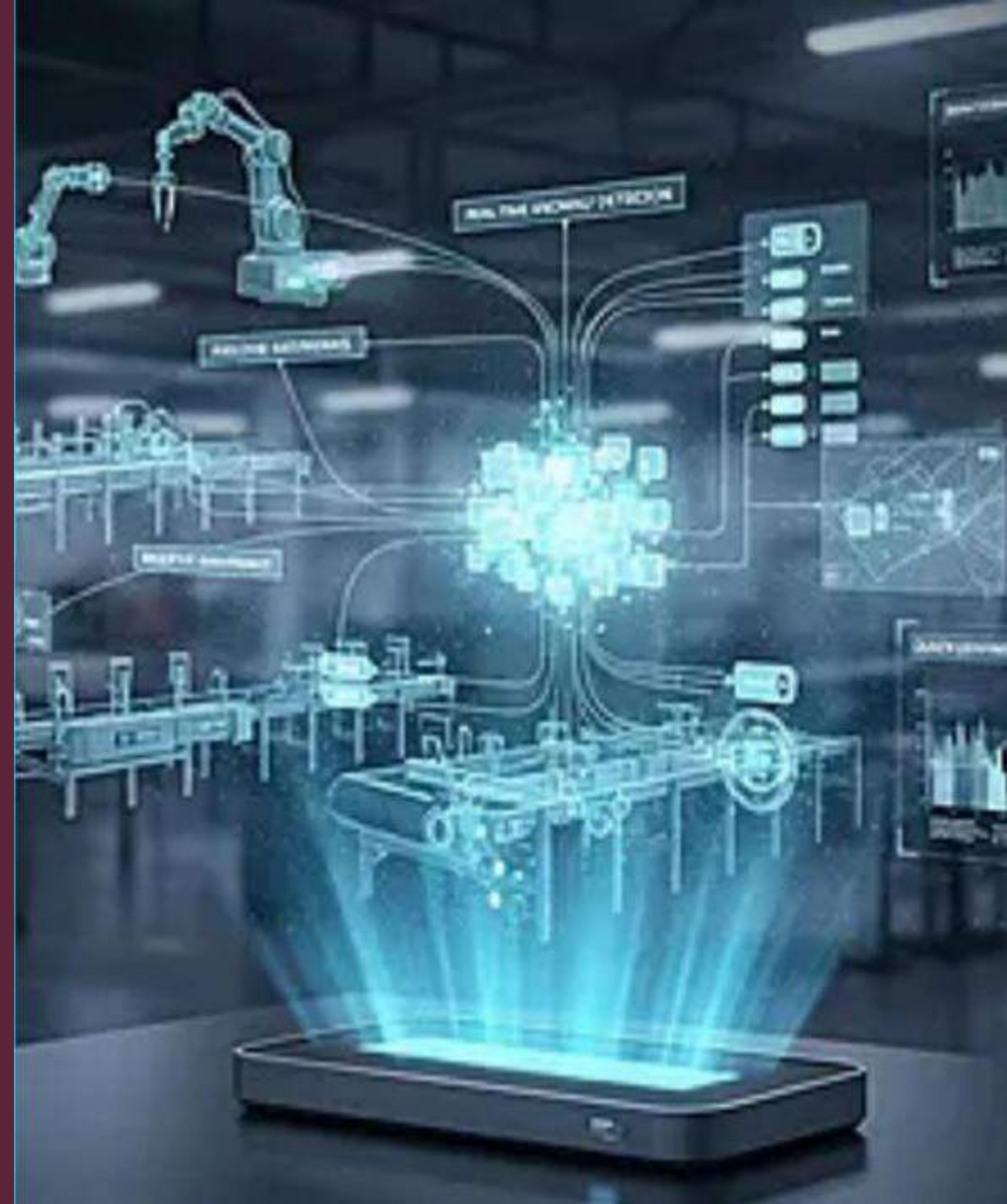
IMMERSIVE VISUALIZATION

LIVE FACTORY FLOOR

The application integrates a live 3D environment using Three.js to mirror real-time machine states.

It features dynamic elements like spinning rotors, pulsating status lights, animated conveyor belts, and ambient particle effects.

This digital twin bridges physical metrics with immersive operational oversight.



MACHINE HEALTH MONITORING

PREDICTIVE MAINTENANCE

Machines dynamically report health parameters from 0-100% in real-time, dictating rotor speeds in the 3D twin.

Strict logic thresholds apply color-coded styling instantly: Healthy (Green), Warning (Amber), and Critical (Red).

A persistent Global Alert Bar aggregates these statuses to immediately flag the lowest performing machine.

The 2D UI panels directly interface with the 3D rendering pipeline for cohesive system monitoring.



3. TELEMETRY & ANALYTICS

Visualizing Live Trends with Chart.js

PRODUCTION & ENERGY TRENDS

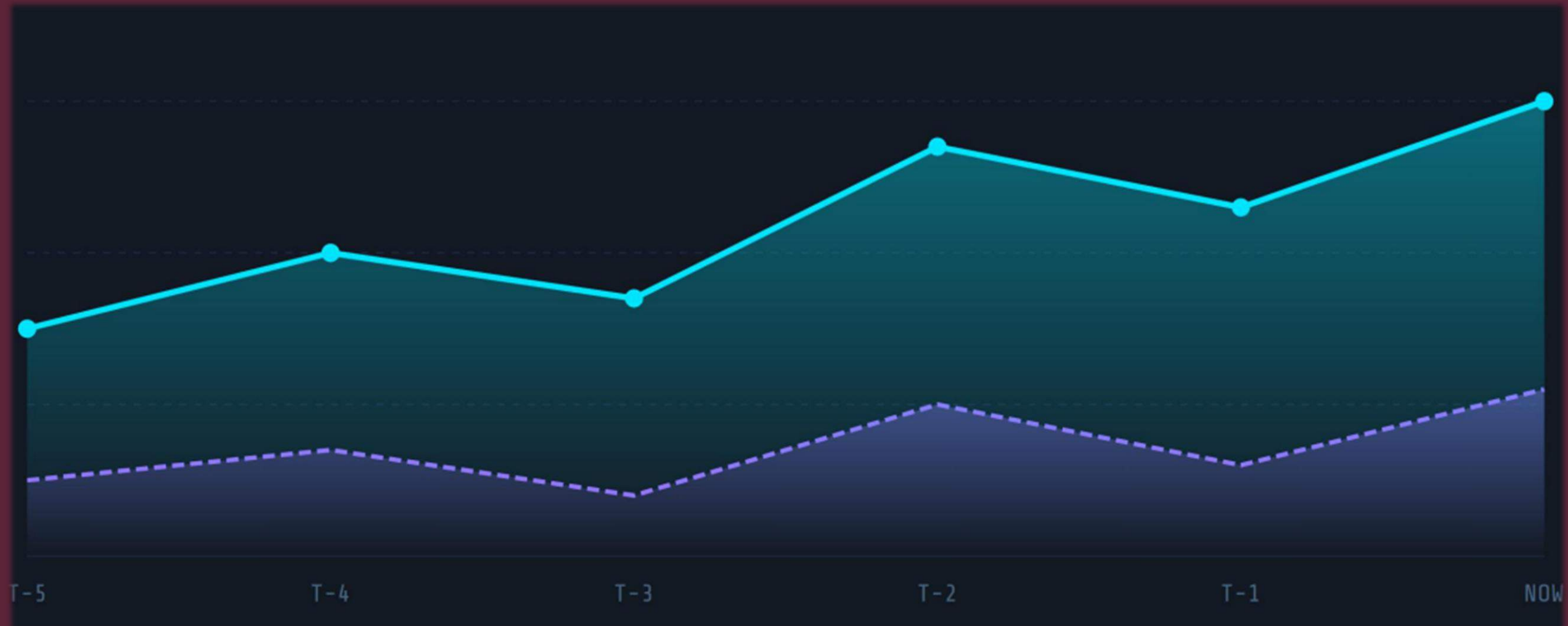
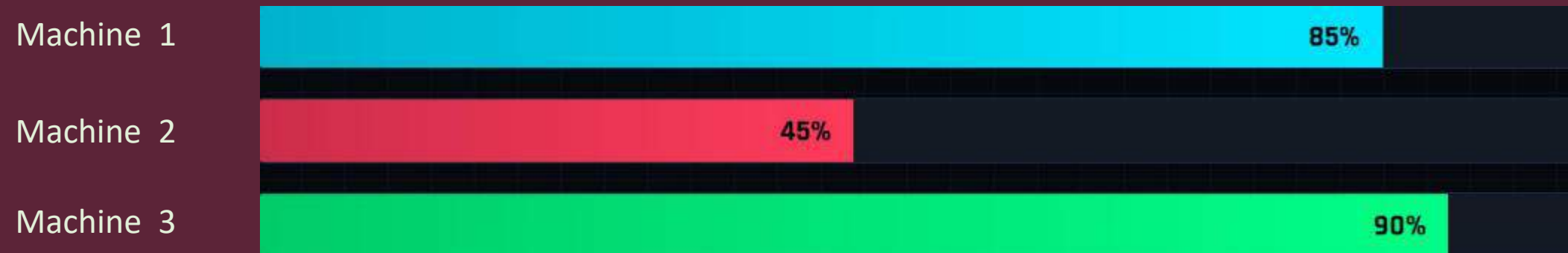


Chart.js powers the dynamic rendering of telemetry data, maintaining a rolling 6-cycle window to visualize the immediate correlation between Production Volume (Cyan) and Energy Expenditure (Purple).

MACHINE UTILIZATION



Granular machine performance is plotted to instantly identify bottlenecks. The chart updates dynamically with system tick, mapping exact values from either the localized simulation engine or a live uploaded dataset.

4. AI-DRIVEN INSIGHTS

Upload, Parse, and Predict with Live Data

ROW-WISE CSV ANALYSIS

INTELLIGENT PARSING

Operators can seamlessly drag-and-drop historical or live batch data via CSV directly into the system.

The engine instantly parses vital logistics parameters such as speed, stock levels, and truck statuses.

A dedicated AI logic layer evaluates metrics row-by-row, predicting and flagging 'Delay Risks' or 'Critical' states.

High-contrast visual status chips make analyzing complex, large-scale datasets instantaneous and scannable.